

# Генерация кода с Source Generators

C# Developer Advanced



Проверить, идет ли запись

**Меня хорошо видно  
& слышно?**





## Дмитрий Гурьянов

Руководитель группы CRM @ SKL Group

- Разрабатываю на .NET с 2015
- Специализация – бизнес приложения на платформе Microsoft

 @guryanovda

 mail@dmgur.ru

# Маршрут вебинара

Знакомство

Введение в Roslyn

Что такое Source Generator

Структура генератора

Live



# Цели вебинара

К концу занятия вы сможете

1. разрабатывать Source Generators для автоматического создания C# кода на этапе компиляции
2. устранять повторяющийся код (boilerplate) и заменять медленную рефлекссию на сгенерированный код
3. применять метапрограммирование для повышения производительности и удобства разработки



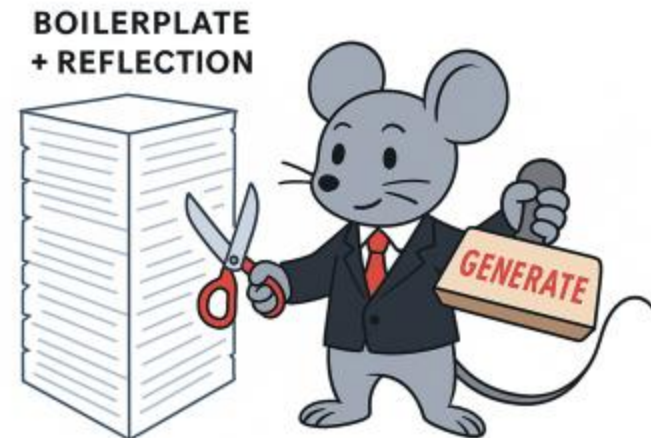
# Зачем нужна генерация кода

---

# Зачем нужна генерация кода

- Проблема boilerplate-кода
- Повторяемость шаблонных конструкций
- Рефлексия как традиционное решение и её ограничения
- Потери производительности
- Сложность поддержки

| Подход     | Недостатки                          |
|------------|-------------------------------------|
| Reflection | Медленно, много runtime-логики      |
| Ручной код | Ошибки и дублирование               |
| T4-шаблоны | Вне компиляции, плохо интегрированы |



# Эволюция кодогенерации в .NET

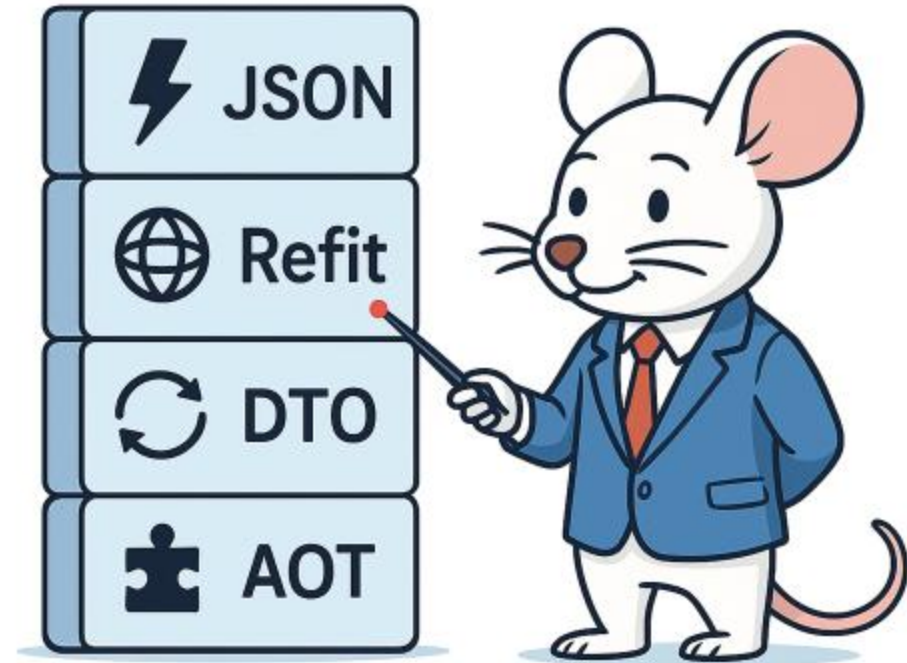
| Технология | Когда использовать        | Проблемы                    |
|------------|---------------------------|-----------------------------|
| Reflection | быстрые прототипы         | медленно, runtime           |
| T4         | генерация текстов         | вне компиляции              |
| IL Emit    | низкоуровневые библиотеки | крайне сложно               |
| SG         | для больших проектов      | нет синхронизации с runtime |





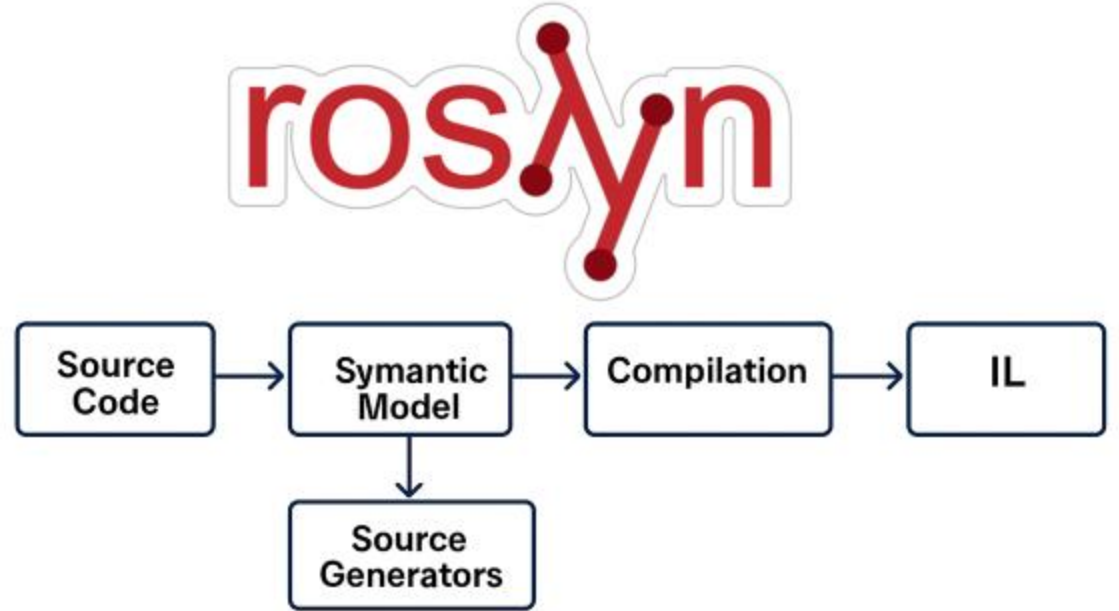
# Где применяются Source Generators

- Быстрая сериализация и десериализация
- Генерация клиентов API
- Автоматический маппинг и DTO
- Обработка атрибутов и конфигурации
- Поддержка DI и AOT-сценариев



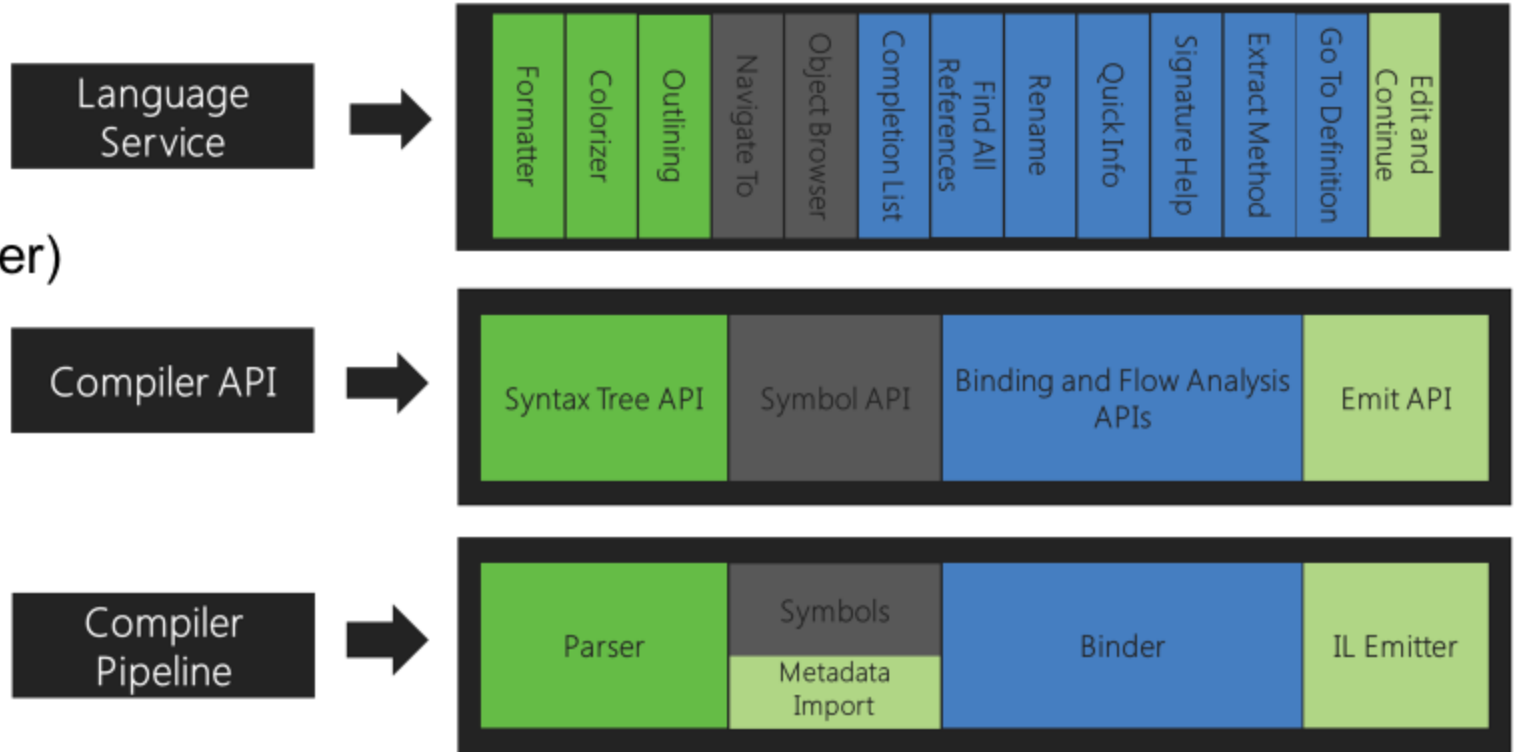
# Roslyn как основа работы Source Generators

- Roslyn — открытый компилятор C# и VB.NET
- Предоставляет API для анализа и генерации кода
- Основные уровни: Syntax, Semantics, Compilation
- Основа для анализаторов, рефакторинга и Source Generators



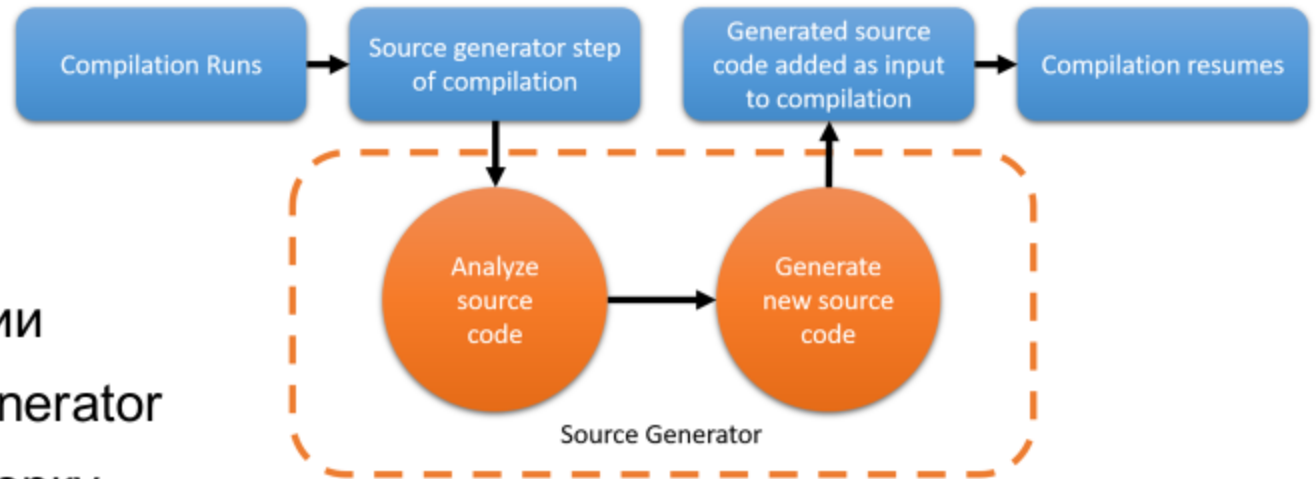
# Пайплайн компиляции в Roslyn

1. Лексический анализ (Lexer)
2. Синтаксический анализ (Parser)
3. Построение Syntax Tree
4. Построение Semantic Model
5. Работа анализаторов и генераторов
6. Генерация IL-кода



# Что такое Source Generator

- Компиляторный плагин Roslyn
- Генерация кода на этапе компиляции
- Реализация интерфейса `ISourceGenerator`
- Добавление новых .cs-файлов в сборку
- Полная типобезопасность
- Без runtime-рефлексии



# Интерфейс ISourceGenerator

```
interface ISourceGenerator
{
    void Initialize(GeneratorInitializationContext context);
    void Execute(GeneratorExecutionContext context);
}
```



# Анализ пользовательского кода в Source Generators

1. Модель проекта представляется в объекте Compilation
2. Отбор узлов для генерации
3. Получение семантической модели
4. Генерация файлов



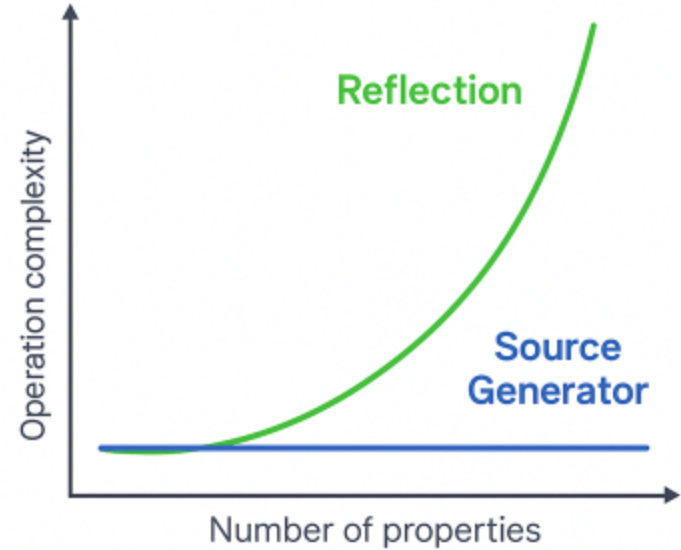
# Как Source Generator добавляет код в компиляцию

1. `context.AddSource(...)`
2. Генерация `SourceText`
3. Виртуальные `.g.cs`-файлы
4. Повторная компиляция с учётом генерации



# Проблемы Рефлексии

- Reflection работает в рантайме и даёт накладные расходы
- Частые сценарии: сериализация, маппинг, DI-контейнеры
- Reflection замедляет холодный старт и увеличивает время отклика
- Source Generators позволяют перенести работу в compile-time



Operation complexity: runtime vs. compile-time



# Отладка Source Generators

1. Запуск отладчика через Debugger.Launch()
2. Отладка через отдельный экземпляр IDE
3. Логирование и ReportDiagnostic
4. Просмотр Generated Files в IDE

```
if (!Debugger.IsAttached)
{
    Debugger.Launch();
}
```



**LIVE**

